



Vel Tech High Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

**Autonomous - Affiliated to Anna University
Avadi, Chennai**

INNOVATIVE ASSIGNMENT PROJECT REPORT

COURSE NAME: DEEP LEARNING ESSENTIALS

COURSE CODE: 21ML53T

1. TITLE

POOLING ANALYSIS FOR DOCUMENT SCANNING

Problem Topic: Pooling

A document-scanning application uses pooling after extracting text edges. Analyze what information is retained and discarded, examine the effect on spatial resolution, and determine whether the chosen pooling operation is appropriate for small characters.

Submitted by:

Name: Harini K

Year: III & SEM: V

Register Number: 113024148027 / VH14121

Department: Computer Science and Engineering (Artificial Intelligence & Machine Learning) & SEC: A

Academic Year: 2026–2027

2. PROBLEM STATEMENT

Document-scanning and Optical Character Recognition systems require the preservation of text boundaries, character strokes, and fine spatial information. After extracting text edges, pooling operations can be used to reduce the size of feature maps and computational complexity.

However, pooling also discards information. This may become problematic when a document contains small or very small characters, since fine strokes and precise spatial locations are important for recognizing text accurately.

The objective of this project is to implement and analyze pooling operations on document edge maps. The project investigates the information retained and discarded by pooling, measures the reduction in spatial resolution, compares Max Pooling and Average Pooling, and evaluates their impact on characters of different sizes.

3. OBJECTIVE

The main objective of this project is to analyze the effect of pooling on text edge information in a document-scanning application.

The specific objectives are:

- To create a sample document image containing different text sizes.
- To preprocess the document image.
- To extract text edges using Canny Edge Detection.
- To implement custom 2×2 Max Pooling with stride 2.
- To implement custom 2×2 Average Pooling with stride 2.
- To compare the outputs of Max Pooling and Average Pooling.
- To measure the reduction in spatial resolution after pooling.
- To analyze information retained and discarded.
- To evaluate the effect of pooling on large, medium, small, and very small

characters.

- To determine whether pooling is appropriate for document-scanning applications involving small text.

4. CONCEPT USED

4.1 Pooling in Deep Learning

Pooling is a downsampling operation commonly used in Convolutional Neural Networks (CNNs). It reduces the spatial dimensions of feature maps while attempting to preserve important information.

A feature map can be represented as: $F \in \mathbb{R}^{H \times W}$ where:

- H represents height
- W represents width

After applying pooling, the output feature map has reduced spatial dimensions.

Pooling provides several advantages:

- Reduces computational complexity
- Reduces memory requirements
- Helps retain important features
- Provides limited robustness to small positional variations

However, pooling also causes information loss.

4.2 Max Pooling

Max Pooling selects the maximum value from each local region.

For a pooling region:

[0 255]

[0 120]

The Max Pooling output is: $\max(0, 255, 0, 120) = 255$

For document edge maps, strong edges generally have high intensity values. Therefore, Max Pooling can preserve strong text boundaries and prominent character strokes.

Information retained:

- Strong edge activations
- Important boundaries
- Prominent strokes
- Dominant local features

Information discarded:

- Exact pixel-level locations
- Weak edge responses
- Fine spatial relationships
- Some thin character strokes

4.3 Average Pooling

Average Pooling calculates the average value of each local region.

For the same region:

[0 255]

[0 120]

The Average Pooling output is: $(0 + 255 + 0 + 120) / 4 = 93.75$

Average Pooling produces a smoother representation.

Information retained:

- General local patterns
- Overall feature intensity
- Average feature distribution

Information weakened or discarded:

- Strong edge responses

- Thin strokes
- Fine boundaries
- Detailed character structure

4.4 Spatial Resolution Reduction

A 2×2 pooling operation with stride 2 reduces both height and width approximately by half.

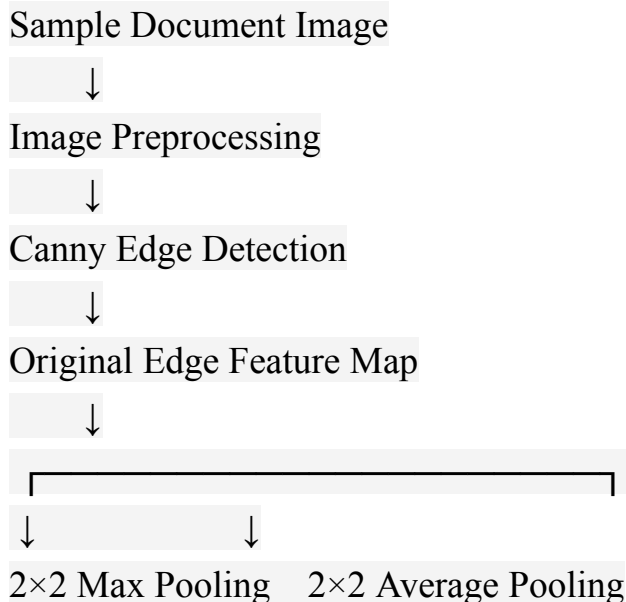
- For an input feature map: 700×1000
- After pooling: 350×500
- Original spatial positions: $700 \times 1000 = 700,000$
- Pooled spatial positions: $350 \times 500 = 175,000$

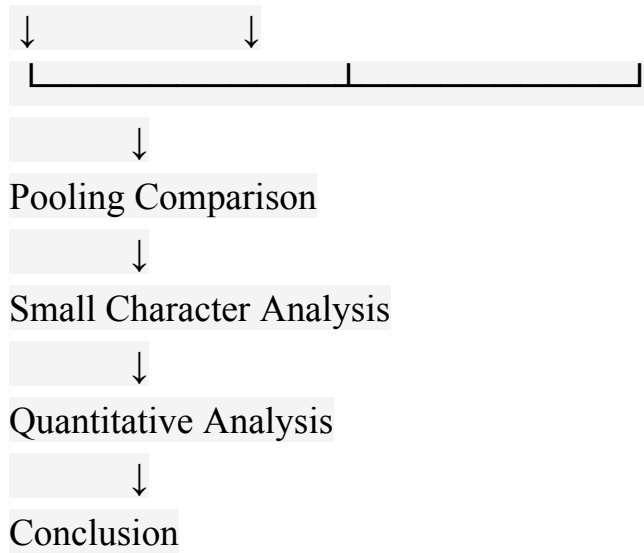
Therefore, spatial reduction is: $(1 - 175000/700000) \times 100 = 75\%$

Thus, 2×2 pooling with stride 2 removes 75% of the spatial positions.

5. METHODOLOGY / WORKING STEPS

The project follows the pipeline below:





Step 1: Dataset Creation

A controlled sample document image was created containing text of different sizes. This enabled the experiment to examine how pooling affects:

- Large text
- Medium text
- Small text
- Very small text

Using a controlled document makes the effect of pooling easier to observe and analyze.

Step 2: Image Preprocessing

The document image was loaded using OpenCV. The preprocessing process included:

- Loading the image
- Converting the image to grayscale
- Applying Gaussian Blur

Grayscale conversion reduces the image from multiple color channels to a single

intensity channel. Gaussian Blur helps reduce noise before edge detection.

Step 3: Edge Detection

Canny Edge Detection was applied to the preprocessed document. The Canny algorithm detects strong intensity transitions and produces an edge map representing:

- Character boundaries
- Text strokes
- Important structural details

The edge map was used as the input feature map for pooling.

Step 4: Max Pooling

A custom implementation of 2×2 Max Pooling with stride 2 was developed using Python and NumPy. Each 2×2 region was processed by selecting its maximum value. This operation preserves strong edge activations while reducing spatial resolution.

Step 5: Average Pooling

A custom implementation of 2×2 Average Pooling with stride 2 was developed. Each 2×2 region was replaced by its average intensity value. This operation creates a smoother feature representation but can weaken strong edges.

Step 6: Pooling Comparison

The following feature maps were compared:

- Original edge map
- Max Pooling output
- Average Pooling output

The comparison included Resolution, Total spatial positions, Number of active features, Active feature percentage, and Mean intensity.

Step 7: Small Character Preservation Analysis

Different regions containing different text sizes were selected. Each region was analyzed before and after Max Pooling. The experiment examined Resolution reduction, Edge preservation, Loss of fine details, and Effect on thin character strokes.

Step 8: Quantitative Analysis

The results were stored in CSV and text files. Graphs were generated to visualize Spatial position reduction and Active feature percentage, providing both visual and numerical evidence.

6. IMPLEMENTATION

6.1 Tools and Libraries

Tool / Library	Purpose
Python	Main programming language
OpenCV	Image processing and edge detection
NumPy	Numerical operations and custom pooling
Matplotlib	Visualization and graphs

Tool / Library	Purpose
Pandas	Metrics table and CSV generation
Git	Version control
GitHub	Repository hosting and commit history
VS Code	Code development environment

6.2 Project Structure

```
pooling-document-analysis/  
|  
|— dataset/  
|   |— sample_document.png  
|  
|— results/  
|   |— preprocessed.png  
|   |— edges.png  
|   |— max_pooling.png  
|   |— average_pooling.png  
|   |— pooling_comparison.png  
|   |— small_character_analysis.png  
|   |— final_metrics.csv  
|   |— spatial_resolution_graph.png
```

```
|   └── feature_retention_graph.png
|
|   └── src/
|       ├── create_dataset.py
|       ├── preprocessing.py
|       ├── edge_detection.py
|       ├── pooling_analysis.py
|       ├── average_pooling.py
|       ├── compare_pooling.py
|       ├── small_character_analysis.py
|       └── final_analysis.py
|
|   └── screenshots/
|
|   └── README.md
|   └── requirements.txt
|   └── .gitignore
```

6.3 Source Code Modules

- **create_dataset.py:** Creates a sample document containing multiple text sizes for experimentation.
- **preprocessing.py:** Performs grayscale conversion and Gaussian Blur.
- **edge_detection.py:** Applies Canny Edge Detection to extract text boundaries.
- **pooling_analysis.py:** Implements custom 2×2 Max Pooling.
- **average_pooling.py:** Implements custom 2×2 Average Pooling.
- **compare_pooling.py:** Compares original, Max Pooled, and Average Pooled feature maps.
- **small_character_analysis.py:** Analyzes the effect of pooling on characters of different sizes.

- **final_analysis.py:** Generates quantitative metrics and graphical visualizations.

6.4 GitHub Repository

GitHub Repository Link:

<https://github.com/kharini118-sys/pooling-document-analysis/tree/main>

The project was developed progressively using meaningful commits such as:

- Initialized project repository
- Added document image dataset
- Added image preprocessing
- Implemented edge extraction
- Implemented 2x2 max pooling
- Implemented 2x2 average pooling
- Added pooling comparison analysis
- Added small character preservation analysis
- Added quantitative pooling analysis
- Polished project documentation and submission files

7. RESULTS & OUTPUT

7.1 Preprocessing Output

The document image was successfully converted into a grayscale image and processed using Gaussian Blur. This prepared the image for reliable edge extraction.

Output: preprocessed.png

DOCUMENT SCANNING

Deep Learning Pooling Analysis

Small characters: ABC123 xyz789

Fine text: 12 34 56 A B C



7.2 Edge Detection Output

Canny Edge Detection successfully extracted text boundaries and character strokes. The resulting edge map contained important structural information required for pooling analysis.

Output: edges.png



7.3 Max Pooling Output

2×2 Max Pooling successfully reduced the feature map dimensions.

Feature Map	Resolution
Original Edge Map	700×1000
Max Pooled Map	350×500

Max Pooling retained prominent edge responses.

Output: max_pooling.png



7.4 Average Pooling Output

Average Pooling also reduced the feature map to 350×500 . However, the edge responses appeared smoother and weaker compared with Max Pooling.

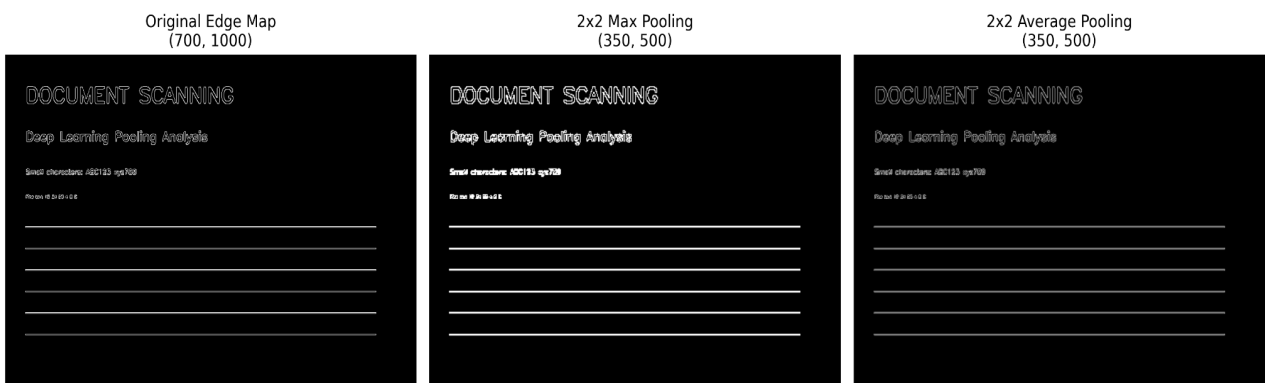
Output: average_pooling.png



7.5 Pooling Comparison

The comparison output displayed: Original edge map, Max Pooling output, and Average Pooling output. This visually demonstrated the differences between the two pooling operations.

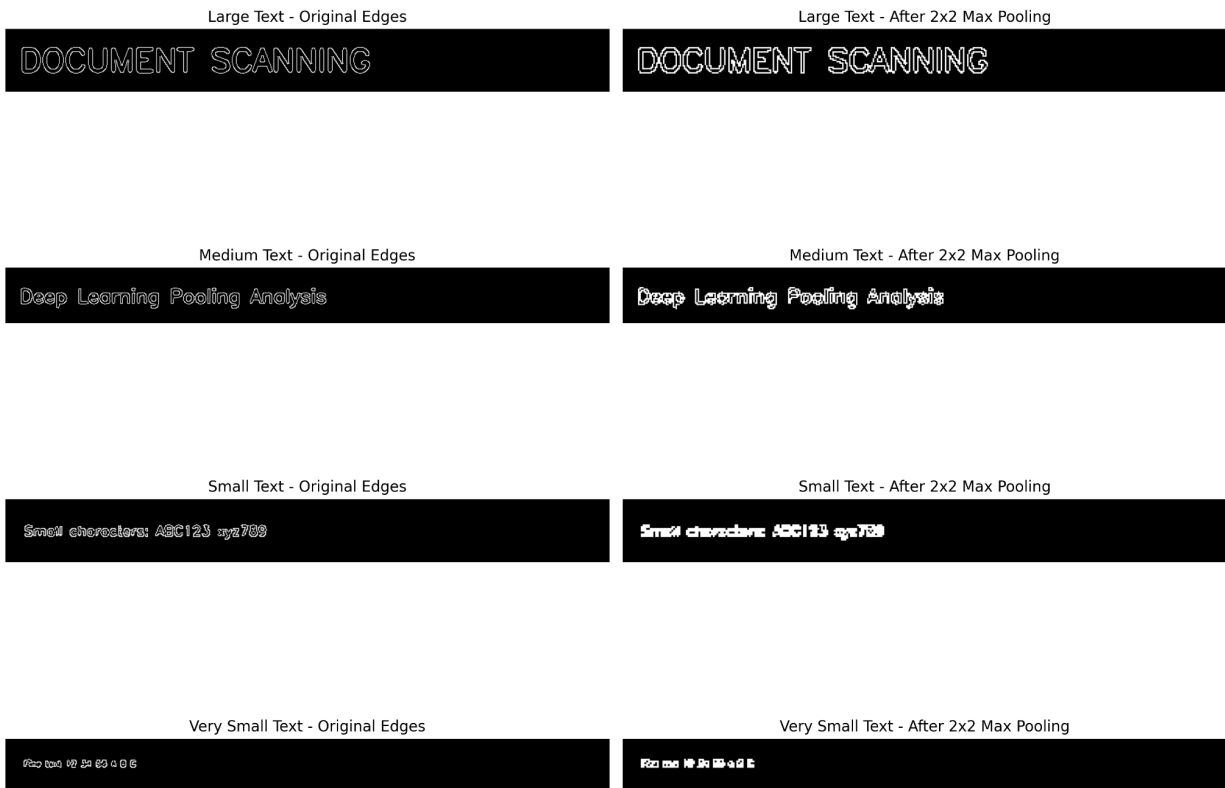
Output: pooling_comparison.png



7.6 Small Character Analysis

The experiment compared text regions containing Large text, Medium text, Small text, and Very small text. The results showed that larger characters retained major structures more effectively. Small and very small characters experienced greater loss of fine spatial information.

Output: small_character_analysis.png



7.7 Quantitative Results

The primary spatial resolution results are:

Method	Resolution	Total Spatial Positions
Original Edge Map	700×1000	700,000
2×2 Max Pooling	350×500	175,000
2×2 Average Pooling	350×500	175,000

Spatial Reduction: 75%

Both pooling methods significantly reduced the number of spatial positions.

8. ANALYSIS

8.1 Information Retained by Max Pooling

Max Pooling retained:

- Strong character edges
- Major boundaries
- Prominent strokes
- High-intensity feature responses

This makes Max Pooling relatively effective when important text features must remain visible after downsampling.

8.2 Information Retained by Average Pooling

Average Pooling retained:

- General feature patterns
- Average local intensity
- Broad structural information

However, averaging reduced the strength of sharp edge responses.

8.3 Information Discarded by Pooling

Both pooling operations discarded spatial information. The following information was reduced or lost:

- Exact feature locations
- Fine pixel-level relationships
- Weak edges
- Thin character strokes
- Detailed spatial structure

Pooling combines multiple input values into a single output value, making the original precise arrangement impossible to fully recover.

8.4 Effect on Spatial Resolution

The original feature map contained 700,000 spatial positions. After 2×2 pooling with stride 2, 175,000 positions remained. Therefore, 75% of spatial positions were removed. This reduction decreases computational cost but also reduces the amount of spatial information available for later processing.

8.5 Effect on Small Characters

The experiment showed a relationship between character size and information loss.

- **Large Characters:** Large characters retained their main boundaries and strokes relatively well.
- **Medium Characters:** Most major structures remained, although fine details were reduced.
- **Small Characters:** Thin strokes and detailed boundaries became less distinct.
- **Very Small Characters:** Very small characters were most vulnerable to pooling because their features occupied only a limited number of pixels. After downsampling, multiple pixels representing fine character details could be merged into a single pooled value.

8.6 Max Pooling vs Average Pooling

Property	Max Pooling	Average Pooling
Strong edges	Better preserved	Weakened
Fine details	Can be lost	Can be blurred
Noise reduction	Limited	Better smoothing
Spatial resolution	Reduced	Reduced
Small character preservation	Relatively better	Less effective
Suitable for text edges	More suitable	Less suitable

Interpretation: For edge-based document analysis, Max Pooling is generally more suitable because it preserves strong edge responses. However, neither pooling method is ideal when very fine character details must be preserved.

9. CONCLUSION

This project successfully implemented and analyzed pooling operations for a document-scanning application. A complete processing pipeline was developed consisting of document creation, preprocessing, edge extraction, Max Pooling,

Average Pooling, small character analysis, and quantitative evaluation.

The experiment demonstrated that pooling significantly reduces spatial resolution. A 2×2 pooling operation with stride 2 reduced the number of spatial positions by 75%.

Max Pooling was found to be more suitable than Average Pooling for preserving strong text edges because it retains the highest activation from each pooling region. Average Pooling produced smoother feature maps but weakened important edge responses.

The small character analysis showed that pooling can negatively affect small and very small text because fine strokes and precise spatial information may be lost during downsampling.

Therefore, the chosen 2×2 Max Pooling operation is suitable for preserving major text edges and reducing computational complexity, but aggressive or repeated pooling is not ideal for document-scanning applications involving very small characters.

For high-accuracy small-text recognition systems, better approaches may include:

- Smaller pooling regions
- Reduced pooling frequency
- Lower stride values
- Higher-resolution feature maps
- Adaptive pooling strategies
- CNN architectures that preserve fine spatial information

Overall, this project demonstrates the important trade-off between computational efficiency and spatial detail preservation in Deep Learning systems.

10. REFERENCES

1. OpenCV Documentation, Image Processing and Canny Edge Detection.
2. NumPy Documentation.

3. Matplotlib Documentation.
4. Pandas Documentation.
5. I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, MIT Press, concepts related to Convolutional Neural Networks and pooling.
6. Course materials and lecture notes for Deep Learning Engineering.